

迭代 4 — AI 工程方法论实战报告

8-1 AI 辅助测试 Prompt 演化实验 -> 8-2 项目级 AGENTS.md -> 8-3 CIVC 自评审计报告

AI 辅助测试 Prompt 演化实验 (回扣 8-1)

案例 1：OrderController#createOrder

初始 Prompt

请帮我给 OrderController 写单元测试，覆盖创建订单接口。

AI 输出问题

- 重言式测试：只断言 200，不断言 success/message。
- 边界缺失：未覆盖 portion 非法、菜品不在明日菜单。
- Mock 不完整：遗漏 UserDao/DishService 协作分支。

改进 Prompt (结构化指令 + Few-shot)

你是资深 Java 测试工程师。请为 Spring Boot 的 OrderController 编写 WebMvcTest，使用 MockMvc + Mockito，不连接数据库。

目标：POST /api/orders (createOrder)

必须覆盖：

- 1) portion 非 whole/half -> success=false, message=份量须为 whole 或 half
- 2) dish.menuDate != tomorrow -> success=false, message=该菜品不在明日可订菜单中
- 3) submitOrder=true -> success=true, message=订单创建成功，且返回 orderId

约束：

- 每个用例必须断言 success 和 message
- 失败分支需要 verify(..., never()) 证明未误调用写操作
- 按如下风格输出：

```
andExpect(jsonPath("$.success").value(false))
andExpect(jsonPath("$.message").value("xxx"))
```

最终可用测试代码

```
package com.canteen.controller;

import com.canteen.dao.UserDAO;
import com.canteen.entity.Dish;
import com.canteen.entity.Order;
```

```
import com.canteen.entity.User;
import com.canteen.service.DishService;
import com.canteen.service.OrderService;
import com.canteen.service.ReportService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(OrderController.class)
class OrderControllerMockTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private OrderService orderService;
    @MockBean
    private ReportService reportService;
    @MockBean
    private DishService dishService;
    @MockBean
    private UserDAO userDAO;

    @Test
    void createOrder_shouldRejectWhenPortionInvalid() throws Exception {
        User student = new User("stu1", "123", "student", "张三");
        when(userDAO.findByUsername("stu1")).thenReturn(student);

        mockMvc.perform(post("/api/orders")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
{"username":"stu1","dishId":"D001","portion":"quarter"}
            """))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.success").value(false))
            .andExpect(jsonPath("$.message").value("份量须为 whole 或 half"));

        verify(dishService, never()).getDishById(any());
        verify(orderService, never()).submitOrder(any());
    }
}
```

```

    }

    @Test
    void createOrder_shouldRejectWhenDishNotInTomorrowMenu() throws Exception {
        User student = new User("stu1", "123", "student", "张三");
        Dish dish = new Dish("D001", "红烧肉", 20.0, "desc", "2099-01-01");
        when(userDAO.findByUsername("stu1")).thenReturn(student);
        when(dishService.getDishById("D001")).thenReturn(dish);
        when(orderService.getTomorrowDate()).thenReturn("2099-01-02");

        mockMvc.perform(post("/api/orders")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
{"username":"stu1","dishId":"D001","portion":"whole"}
"""))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.success").value(false))
            .andExpect(jsonPath("$.message").value("该菜品不在明日可订菜单中"));

        verify(orderService, never()).submitOrder(any());
    }

    @Test
    void createOrder_shouldReturnSuccessWhenSubmitSucceeded() throws Exception {
        User student = new User("stu1", "123", "student", "张三");
        Dish dish = new Dish("D001", "红烧肉", 20.0, "desc", "2099-01-02");
        Order order = new Order();
        order.setOrderId("01001");
        order.setUsername("stu1");

        when(userDAO.findByUsername("stu1")).thenReturn(student);
        when(dishService.getDishById("D001")).thenReturn(dish);
        when(orderService.getTomorrowDate()).thenReturn("2099-01-02");
        when(orderService.createOrder(student, dish, "whole")).thenReturn(order);
        when(orderService.submitOrder(order)).thenReturn(true);

        mockMvc.perform(post("/api/orders")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
{"username":"stu1","dishId":"D001","portion":"whole"}
"""))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.success").value(true))
            .andExpect(jsonPath("$.message").value("订单创建成功"))
            .andExpect(jsonPath("$.order.orderId").value("01001"));
    }
}

```

案例 2：AdminDishController#save/#delete

初始 Prompt

给管理员菜品接口写几个测试。

AI 输出问题

- 仅 happy path，未覆盖权限分支。
- 未覆盖 `menuDate` 非法格式、负价格、空字段。
- 未覆盖“关联订单不可删除”的业务约束。

改进 Prompt (CoT 分步骤 + 角色扮演)

你扮演“代码评审 + 测试设计师”。
先按分支清单设计，再输出最终测试代码（不输出思维链）。

模块：AdminDishController (WebMvcTest + MockBean)

必须覆盖：

Step1 非 admin 查询列表失败：message=无权限管理菜品

Step2 save 入参校验：menuDate 非 yyyy-MM-dd 失败

Step3 save 数据清洗：id/name/menuDate trim, description null -> ""

Step4 delete 业务约束：countByDishId > 0 时失败

Step5 delete 成功分支：success=true

规则：

- 每个测试断言 success + message (或关键字段)
- 至少一个失败分支断言 never()

最终可用测试代码

```
package com.canteen.controller;

import com.canteen.dao.OrderDAO;
import com.canteen.dao.UserDAO;
import com.canteen.entity.Dish;
import com.canteen.entity.User;
import com.canteen.service.DishService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;
```

```

import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest(AdminDishController.class)
class AdminDishControllerMockTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserDao userDao;
    @MockBean
    private DishService dishService;
    @MockBean
    private OrderDAO orderDAO;

    @Test
    void list_shouldRejectWhenNotAdmin() throws Exception {
        when(userDao.findByUsername("stu1")).thenReturn(new User("stu1", "123",
"student", "张三"));

        mockMvc.perform(get("/api/admin/dishes").param("username", "stu1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.success").value(false))
            .andExpect(jsonPath("$.message").value("无权限管理菜品"));

        verify(dishService, never()).getAllDishes();
    }

    @Test
    void save_shouldRejectWhenDateInvalid() throws Exception {
        when(userDao.findByUsername("admin")).thenReturn(new User("admin", "123",
"admin", "管理员"));

        mockMvc.perform(post("/api/admin/dishes")
            .param("username", "admin")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"id":"D001","name":"鱼香肉
丝","price":18.5,"description":"desc","menuDate":"2099/01/02"}
                """))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.success").value(false))
            .andExpect(jsonPath("$.message").value("就餐日须为 yyyy-MM-dd 格
式"));
    }

```

```

        verify(dishService, never()).saveDish(any());
    }

    @Test
    void save_shouldTrimFieldsAndSucceed() throws Exception {
        when(userDAO.findByUsername("admin")).thenReturn(new User("admin", "123",
"admin", "管理员"));
        when(dishService.saveDish(any(Dish.class))).thenReturn(true);

        mockMvc.perform(post("/api/admin/dishes")
            .param("username", "admin")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"id":" D002 ","name":" 糖醋里脊
", "price":20.0, "description":null, "menuDate":" 2099-01-03 "}
                """))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.success").value(true))
            .andExpect(jsonPath("$.message").value("保存成功"))
            .andExpect(jsonPath("$.dish.id").value("D002"))
            .andExpect(jsonPath("$.dish.name").value("糖醋里脊"))
            .andExpect(jsonPath("$.dish.menuDate").value("2099-01-03"))
            .andExpect(jsonPath("$.dish.description").value(""));
    }

    @Test
    void delete_shouldRejectWhenDishHasOrders() throws Exception {
        when(userDAO.findByUsername("admin")).thenReturn(new User("admin", "123",
"admin", "管理员"));
        when(orderDAO.countByDishId("D003")).thenReturn(2);

        mockMvc.perform(delete("/api/admin/dishes/D003")
            .param("username", "admin"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.success").value(false))
            .andExpect(jsonPath("$.message").value("该菜品已有 2 条关联订单，无
法删除。可修改供餐日或保留记录。"));

        verify(dishService, never()).deleteDishById(any());
    }

    @Test
    void delete_shouldReturnSuccessWhenRemoved() throws Exception {
        when(userDAO.findByUsername("admin")).thenReturn(new User("admin", "123",
"admin", "管理员"));
        when(orderDAO.countByDishId("D004")).thenReturn(0);
        when(dishService.deleteDishById("D004")).thenReturn(true);

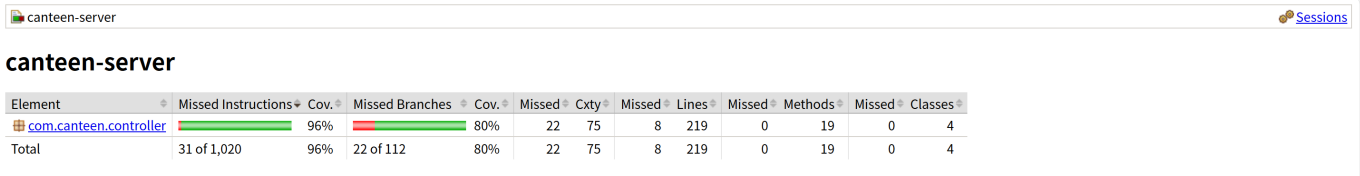
        mockMvc.perform(delete("/api/admin/dishes/D004")
            .param("username", "admin"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.success").value(true))
            .andExpect(jsonPath("$.message").value("已删除"));
    }

```

```
}  
}
```

覆盖率截图（核心 API 模块）

- 覆盖率报告：[canteen-server/target/site/jacoco/index.html](#)
- 核心 API ([com.canteen.controller](#))：Lines 96%, Branches 80%



项目级 AGENTS.md（回扣 8-2）

0) 快速上手（先看这一段）

- 这是一个前后端分离的校园订餐系统：[frontend](#)(Vue3) + [canteen-server](#)(Spring Boot 3 + MyBatis)。
- 核心链路：登录 -> 学生看明日菜单/下单 -> 管理员维护菜品与查看统计。
- 后端分层固定：[controller](#) -> [service](#) -> [dao](#) -> [mapper/sql](#)，禁止跨层直连。
- 新功能优先补 [Mock](#) 单测（控制层 [WebMvcTest](#)，业务层 [Mockito](#)）。
- 提交前至少保证：编译通过、已有测试通过、接口返回结构与现有风格一致。

1) 项目架构概述

- 架构：前后端分离 + 后端经典三层（Controller/Service/DAO）。
- 前端通过 HTTP 调用后端 [/api/**](#) 接口，不直接访问数据库。
- 后端通过 MyBatis DAO + XML mapper 访问 MySQL。
- 业务核心围绕“明日订餐”：日期、角色、权限、订单状态约束必须显式校验。

2) 目录结构说明

- [frontend/](#): Vue3 前端工程
- [canteen-server/src/main/java/com/canteen/controller](#): API 控制层
- [canteen-server/src/main/java/com/canteen/service](#): 业务逻辑层
- [canteen-server/src/main/java/com/canteen/dao](#): MyBatis DAO 接口
- [canteen-server/src/main/java/com/canteen/entity](#): 实体模型
- [canteen-server/src/main/resources/mappers](#): MyBatis XML
- [canteen-server/src/test/java/com/canteen](#): 后端单元测试
- [docs/](#): 课程报告、实验文档、截图证据

3) 核心模块职责

- [LoginController/LoginService](#): 用户登录鉴权，返回用户角色([student/admin](#))。
- [DishController/DishService](#): 学生侧“明日菜单”查询。
- [OrderController/OrderService](#): 下单、个人订单、取消明日订单、份量更新。

- **AdminDishController**: 管理员菜品增删改查，删除前做订单关联检查。
- **ReportService**: 管理端统计报表（人数、订单量、菜品需求量）。

4) 渐进式工作流程（Anthropic 风格）

1. **先理解上下文**：先读目标模块的 Controller + Service + 相关测试。
2. **再做最小改动**：只改与需求直接相关文件，避免顺手重构无关代码。
3. **同步补测试**：每新增/修改业务分支，都要新增对应断言。
4. **最后统一验证**：本地运行测试，检查覆盖率与关键返回字段。

5) 编码规范约束

- Java 17 语法，沿用现有命名与分层，不引入新架构模式。
- Controller 只做参数校验、权限判断、响应组装；复杂逻辑放 Service。
- 返回值保持现有风格：**Map<String, Object>** + **success/message/...** 字段。
- 错误信息使用现有中文文案风格，避免中英文混杂。
- DAO 只声明数据访问；SQL 逻辑写在 mapper XML，不在 Java 拼接 SQL。
- 新增接口必须覆盖：正常路径 + 至少一个边界/失败路径。
- 测试优先：
 - Controller：**@WebMvcTest** + **@MockBean** + **MockMvc**
 - Service：**MockitoExtension** + mock DAO
- 不修改 **target/** 产物，不把构建产物当源码维护。

6) 禁止操作清单

- 禁止跨层调用（如 Controller 直接调 DAO）。
- 禁止绕过权限校验（学生/管理员角色判断不可省略）。
- 禁止删除或弱化已有业务约束（如“仅可取消明日订单”）。
- 禁止提交密钥、数据库密码、**.env** 等敏感信息。
- 禁止使用破坏性 Git 操作（**reset --hard**、强推）处理常规任务。
- 禁止在无测试验证情况下大规模改动核心 API。

7) 交付自检清单

- 改动是否仅影响目标需求范围？
- 接口字段是否与前端约定兼容（**success/message/data**）？
- 关键分支是否有测试覆盖（至少成功+失败）？
- **canteen-server** 测试是否通过，覆盖率是否满足当前任务要求？

CIVC 自评审审计报告（回扣 8-3）

审计对象

- 项目：**canteen-sys**
- 框架：CIVC 四阀门（Constraint / Information / Verification / Correction）
- 审计基线：当前仓库代码、**README.md**、**AGENTS.md**、现有测试与覆盖率流程

总体结论

- **Constraint**：中等偏弱（可读写范围较大，缺少硬沙盒）
 - **Information**：中等偏强（已有项目级上下文文档）
 - **Verification**：较强（自动化测试 + 覆盖率门槛已落地）
 - **Correction**：中等偏弱（有 Git 可回退，但缺少“一键回滚”标准机制）
-

A. Constraint（约束）

当前状态

- AI 可访问并修改仓库内大多数文件（如 `frontend/`、`canteen-server/`、`docs/`）。
- 已有软约束文档：`AGENTS.md`（分层规范、禁止操作、测试优先）。
- 当前未配置文件级硬隔离（如仅允许改 `src/**`、禁止改 `main` 受保护文件）。

风险判断

- 若 Prompt 不精确，AI 可能越界修改无关模块。
- 构建产物（`target/`）可能被误纳入版本管理。

改进方案（薄弱阀门）

1. 分支隔离：强制 AI 相关改动在 `feat/ai-*` 分支完成，禁止在主分支直改。
 2. 路径白名单：在任务说明中固定可改路径（如仅 `canteen-server/src/**` 与 `docs/**`）。
 3. 提交前阻断：新增 pre-commit 检查，禁止提交 `target/**`、密钥文件、数据库凭据。
 4. 高风险文件保护：对 `application.properties`、`db.sql` 采用“必须人工确认”策略。
-

B. Information（告知）

当前状态

- 已有 `README.md`：架构、目录、模块职责、运行方式、CI 说明。
- 已有项目级 `AGENTS.md`：渐进式披露、分层边界、编码规范、禁令清单。
- 能支撑“新 AI 助手快速理解并产出代码”。

风险判断

- 业务约束文案分散（`README`、代码注释、测试中都有），单点事实源不够集中。
- 接口字段契约缺少统一 API 清单文档。

改进方案（薄弱阀门）

1. 单点上下文入口：在 `AGENTS.md` 新增“必须先读文件清单”（`README` + 核心 Controller + 测试）。
 2. 接口契约文档化：新增 `docs/api-contract.md`，统一字段与错误文案。
 3. 业务规则卡片化：新增 `docs/business-rules.md`，沉淀“仅可取消明日订单”等硬规则。
 4. 任务模板化：为 AI 任务提供固定模板（目标、可改路径、验收标准、禁止项）。
-

C. Verification（验证）

当前状态

- 后端有单元测试：`canteen-server/src/test/java/**`。
- 已接入 JaCoCo，核心 API (`com.canteen.controller`) 覆盖率门槛 `LINE >= 80%`。
- 现状实测：`Lines 96%、Branches 80%` (截图：`docs/images/coverage-core-api-8-1.png`)。
- 本地可通过 `mvn clean test` 自动触发测试 + 覆盖率检查。

风险判断

- 当前验证重心在后端；前端缺少同等级自动化测试门槛。
- 对“接口回归兼容性”缺少契约测试 (仅靠单元测试与人工联调)。

改进方案 (薄弱阀门)

1. **CI 强制门禁**：PR 必须通过 `mvn clean test` 与覆盖率检查才可合并。
2. **前端补测**：引入 Vue 组件/页面关键流程测试 (登录、下单、管理员操作)。
3. **契约测试**：增加接口契约测试，锁定 `success/message/data` 字段兼容性。
4. **变更影响清单**：每次 AI 改动附“影响接口 + 新增测试点”表格。

D. Correction (纠正)

当前状态

- 具备 Git 历史，可人工回退。
- 但没有项目约定的“一键回滚”脚本与流程。
- 出错后依赖开发者手工执行多步命令，恢复成本较高。

风险判断

- 紧急修复场景下，回滚不够标准化，容易误操作。
- 多文件改动时，人工挑选回退范围容易遗漏。

改进方案 (薄弱阀门)

1. **一键回滚脚本**：新增 `scripts/rollback-last-change.(ps1|sh)`，回退最近一次 AI 提交并打印影响文件。
2. **最小提交策略**：AI 改动按功能拆分小提交，降低回滚粒度成本。
3. **失败自动止损**：CI 失败自动阻止合并，并提示回滚命令模板。
4. **回滚演练**：每个迭代至少一次“故障-回滚-恢复”演练并记录耗时。

审计结论与优先级

- **P0 (立即)**：Constraint 的路径白名单 + pre-commit 阻断；Correction 的一键回滚脚本。
- **P1 (本迭代)**：Information 的 API 契约与业务规则文档。
- **P2 (下迭代)**：Verification 扩展到前端自动化与契约测试。

执行以上改进后，项目将从“可用的 AI 协作”提升为“可控、可证、可恢复的 AI 协作”。